

What Is System Design (HLD vs LLD)

Phase 1 · Fundamentals · Estimated time: 1 hour

1. Concept From First Principles

You already design systems every day — you just haven't called it that. When you build a Laravel application, you decide things like: should this run as a background job or inline? Should we cache this expensive query in Redis? Should file uploads go straight to disk or to S3? Every one of those decisions is a small act of system design. High-Level Design (HLD) is the same kind of thinking, done deliberately, at the scale of an entire product, before a single line of code is written.

Formally: **System design** is the process of defining the architecture, components, data flow, and interfaces of a system to satisfy specified functional and non-functional requirements (scale, availability, latency, cost). It sits above code — it answers *what pieces exist and how they talk to each other*, not *what does each function do internally*.

HLD vs LLD — the actual difference

High-Level Design (HLD) is the architecture blueprint: which services exist, which databases they use, how they communicate (REST? a queue?), where the bottlenecks will be, and how the system behaves under load or failure. Deliverables look like box-and-arrow diagrams, API contracts, and capacity estimates.

Low-Level Design (LLD) zooms into a single box from the HLD diagram and asks: what classes, functions, design patterns, and database tables actually implement this? If you were designing a parking lot system, the HLD says "there is a Booking Service backed by a relational database"; the LLD says "here is the **ParkingSpot** class, the **Vehicle** class, and the strategy pattern used to price a ticket."

Backend engineer analogy

Think of HLD as the decision to use a queue (RabbitMQ/SQS) so your Laravel app doesn't block on sending emails synchronously. LLD is then writing the actual **Job** class, its **handle()** method, and retry logic. You've been doing LLD-adjacent work for 9 years. HLD asks you to zoom out one level further and reason about the whole system, often for a product you've never built, in 30–45 minutes, out loud.

A third, related term you'll hear: **coding interviews** test whether you can write correct, efficient code for a well-defined problem (e.g., "reverse a linked list"). HLD interviews test whether you can take an *ambiguous*, large problem ("design WhatsApp") and produce a workable architecture while explaining trade-offs. These are different muscles, and most 9-year backend engineers have only trained the first one.

Here's the mistake almost every strong coder makes on their first HLD attempt: they start designing the database schema or writing pseudo-code within the first two minutes, because that's the muscle memory from years of feature work. An HLD interview rewards the opposite instinct — stay at the architecture altitude deliberately, resist the pull toward implementation detail, and only zoom in when

the interviewer asks you to or when a specific component genuinely needs justification (e.g., "why Kafka and not a simple queue here"). Staying too low too early is the single most common reason otherwise-strong engineers get weak HLD interview feedback despite having perfectly good final answers.

It also helps to understand *why* companies weight HLD so heavily for backend roles specifically. A backend engineer with 9 years of experience is expected to eventually own architecture decisions, not just implement tickets — things like "should this new feature be its own service or live inside the monolith," "what happens to this flow if the payment gateway times out," or "will this design still work at 10x our current user base." HLD interviews are a proxy for exactly that judgment, compressed into 30–45 minutes with an unfamiliar problem, which is also why interviewers care more about your reasoning process than about arriving at any single "correct" architecture.

2. Real-World Example / Case Study

Consider Swiggy's order-placing flow. At the **HLD** level, the architecture is described as a set of cooperating services: a Restaurant Service (menu, availability), an Order Service (cart, checkout), a Payment Service (talks to a payment gateway), a Delivery-Partner Tracking Service (live location), and a Notification Service (SMS/push). The HLD conversation is about how these services communicate (synchronous REST calls vs. asynchronous events), which data each one owns, and how the system stays available when, say, the Payment Service is slow.

Zoom into just the Order Service and you're now in **LLD** territory: an **Order** class with a state machine (CREATED → CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED), a repository pattern for persistence, and validation logic for cart items. Razorpay's public system design write-ups follow the same split: architecture diagrams for how their payment gateway routes a transaction across banks (HLD), separate from the internal class design of their reconciliation engine (LLD).

This distinction is exactly why interview processes at Flipkart, PhonePe, Razorpay, Careem, and Noon-tier companies run *separate* HLD and LLD rounds for senior backend roles — they are testing different skills, and this program focuses entirely on the HLD skill.

It's worth noting how this plays out slightly differently across the regions you're preparing for. Indian product companies (Flipkart, Swiggy, Meesho-tier) commonly run a dedicated HLD round for SDE-2 and above, often 45 minutes, expecting you to reach a defensible architecture with trade-offs stated explicitly. Gulf-based product and fintech teams (Careem, Noon, Talabat-tier) frequently blend a shorter HLD discussion into a broader "system thinking" round, sometimes paired with a whiteboard walkthrough of a system you've actually built — so being able to describe your own past architecture decisions in HLD language is just as valuable as solving a fresh prompt. Phase 4, Chapter 2 covers these regional differences in full depth once you have the vocabulary to appreciate them.

3. Diagram

HLD view of a checkout flow (the level of detail an HLD interview expects):



HLD stops here: boxes, data stores, and the arrows between them.

LLD view zooms into just the "Order Service" box:

```

class Order {
    id, userId, items[], status, totalAmount
    transitionTo(newStatus) // enforces valid state transitions
}

class OrderRepository {
    save(order)
    findById(id)
}

class OrderService {
    createOrder(cart) -> Order
    cancelOrder(id)
}

LLD goes here: class fields, methods, patterns, DB schema detail.

```

4. Trade-offs Table

Aspect	HLD	LLD
Primary question	What components exist and how do they interact?	How is each component implemented internally?
Typical deliverable	Architecture diagram, API contracts, capacity plan	Class diagrams, method signatures, DB schema
Time horizon in interview	30–45 min, whole system	30–45 min, one module in depth
Skills tested	Trade-off reasoning, estimation, scale awareness	OOP design, design patterns, clean code
Example question	"Design WhatsApp"	"Design a parking lot / rate limiter class"
Who asks it more	Product companies, senior/staff roles	Product & service companies, all levels

5. Common Interview Questions

Q1: "What is the difference between HLD and LLD?"

Model approach: Define both in one sentence each, then give a concrete example splitting a system (e.g., "URL shortener" HLD = load balancer + app servers + key-generation service + DB; LLD = the actual base-62 encoding algorithm and its class). Mention that HLD deliverables are diagrams and trade-offs, LLD deliverables are class-level designs. Keep it under 90 seconds — this is usually a warm-up question.

Q2: "How would you approach this differently if it were an LLD round?"

Model approach: Say you'd stop drawing boxes for external services and instead pick one core component, then design its classes, interfaces, and design patterns (e.g., Strategy for pricing, Observer for notifications), plus the exact DB schema for that component only.

Q3: "Is this an HLD or LLD question?" (interviewer deliberately ambiguous)

Model approach: This tests whether you ask clarifying questions instead of guessing. The correct move is to ask the interviewer directly: "Should I focus on overall architecture and scale, or on class-level design?" Asking this signals seniority — it is a legitimate, expected question, not a weakness.

Q4: "Give an example of an HLD decision you made in a real project, and what would have happened if you'd chosen differently."

Model approach: Pick a real decision from your 9 years of experience — e.g., choosing to introduce a queue instead of a synchronous call, or splitting a monolith endpoint into its own service. State the context, the alternative you didn't choose, and the concrete consequence you were avoiding (e.g., "if we'd kept it synchronous, a slow third-party SMS provider would have blocked every signup request"). Interviewers weight real, lived trade-off reasoning highly — it's harder to fake than a rehearsed answer.

6. Practice Exercises

- 1 Take "Design an online food ordering system." Write two separate lists: one of decisions that belong to HLD (e.g., "use a message queue for order notifications") and one that belong to LLD (e.g., "Order class has a status enum"). Aim for at least 6 items in each list.
- 2 Sketch an HLD-only box diagram (boxes and arrows, no class details) for a URL shortener, using only what you know today — you'll revisit and correct this in Chapter 15's recap.
- 3 Explain out loud, in under 2 minutes, why a company would run both an HLD interview and a separate LLD interview for the same candidate. Record yourself if possible — verbalizing is a different skill from reading.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"HLD Vs LLD System Design Essentials"	Direct, focused comparison with examples, good first watch.
English	"High Level Design vs Low Level Design HLD vs LLD System Design Concepts"	Reinforces the distinction with a second explanation style.

Language	Video & Channel	Why it helps
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp (Shradha Khapra)	Popular Hindi channel; frames HLD/LLD inside a full prep roadmap.
Hindi	"HLD vs LLD in System Design Explained Simply!" (Short)	Quick Hindi recap if you want a 60-second refresher.

8. Key Takeaways

- System design is a discipline you already practice informally; HLD makes it deliberate and applies it to a whole product.
- HLD = architecture & component interaction. LLD = internal implementation of one component.
- HLD deliverables are diagrams, API contracts, and capacity estimates — not code.
- Companies test HLD and LLD in separate rounds because they test different skills.
- If a question is ambiguous about scope, ask the interviewer whether they want HLD or LLD depth — this is expected, not a weakness.
- This entire program is scoped to HLD; LLD (class design, patterns) is intentionally out of scope.